

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
 Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	Assessment:	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.		
Student Name:	Sasikumar Megha	Reg. No.:	192471038
Section / Batch:	3 rd CS & BS	Date:	

Instructions to Students

- Answer the following question with proper pseudocode and logical explanation.
- Clearly specify the algorithmic approach and complexity class wherever applicable.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) for complexity analysis.
- Include state-space representation, recursion steps, and pruning conditions in backtracking problems.
- Neat presentation and step-by-step justification are mandatory.

Question Type	Focus Area	Questions
Pseudocode Writing Task	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – Pseudocode Writing Task

(95) ✓

Code of Conduct

I, Sasikumar Megha (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Megha

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	✓			
Algorithm Design	30%	✓			
Use of Control Structures	20%	✓			
Pseudocode Structure	10%	✓			
Completeness	20%		✓		
Total Marks (100):		95			

Faculty In-charge	Course Coordinator	Head of Department
Name: <u>K. Suleen</u>	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: <u>31/8/26</u>	Date: _____	Date: _____

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA06

COURSE OUTCOME COVERED

CO5: Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.

(BL5,BL6)

Assessment Tool Weightage: 20%

QUESTIONS: 10 Total Marks: 50

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Pseudocode Writing Task (Algorithm Design)

Q90. Bin Packing Approximation Using Best-Fit Strategy

Idea: For each item, place it in the bin having the **least remaining space** that can still fit the item.

Pseudocode

BEST_FIT(items, capacity)

Input: items[] → sizes of items

capacity → capacity of each bin

Output:

bins[] → bins containing assigned items

1. bins ← empty list
2. FOR each item in items DO
3. bestBin ← NONE
4. minRemaining ← INFINITY
5. FOR each bin in bins DO
6. remaining ← capacity - bin.currentSize
7. IF remaining $\geq$ item THEN
8. IF remaining - item < minRemaining THEN
9. minRemaining ← remaining - item
10. bestBin ← bin
11. END IF
12. END IF
13. END FOR
14. IF bestBin $\neq$ NONE THEN
15. Add item to bestBin
16. bestBin.currentSize ← bestBin.currentSize + item
17. ELSE
18. Create a new bin

```

19.      Add item to the new bin
20.      newBin.currentSize ← item
21.      Add newBin to bins
22.      END IF
23.      END FOR

24.      RETURN bins

```

Complexity: $O(n \times b)$ where n = number of items and b = number of bins.

Key point: Best-Fit tries to **minimize the unused space** in the selected bin.

Q91. N-Queens Problem with Symmetry Elimination

Idea: Place queens row by row. To avoid mirror-image duplicate solutions, restrict the queen in the **first row to half of the columns**, then generate the remaining symmetric solutions.

Pseudocode

N_QUEENS(N)

Input:

N → size of chessboard

Output: all unique solutions without symmetric
duplicates

```

1.      board[1..N] ← EMPTY
2.      solutions ← empty list

3.      FOR col ← 1 TO floor(N / 2) DO
4.      IF SAFE(1, col) THEN
5.      board[1] ← col
6.      BACKTRACK(2)
7.      END IF
8.      END FOR

9.      IF N is odd THEN

```

10. $\text{col} \leftarrow (N + 1) / 2$
11. IF SAFE(1, col) THEN
12. $\text{board}[1] \leftarrow \text{col}$
13. BACKTRACK(2)
14. END IF
15. END IF

16. RETURN solutions

BACKTRACK(row)

1. IF row > N THEN
2. Add board to solutions
3. RETURN 4. END IF

5. FOR col $\leftarrow$ 1 TO N DO
6. IF SAFE(row, col) THEN
7. $\text{board}[\text{row}] \leftarrow \text{col}$
8. BACKTRACK(row + 1)
9. $\text{board}[\text{row}] \leftarrow \text{EMPTY}$
10. END IF
11. END FOR

SAFE(row, col)

1. FOR i $\leftarrow$ 1 TO row - 1 DO
2. IF $\text{board}[i] = \text{col}$ THEN
3. RETURN FALSE
4. END IF

5. IF $|\text{board}[i] - \text{col}| = |i - \text{row}|$ THEN

6. RETURN FALSE
7. END IF
8. END FOR

9. RETURN TRUE

Pruning: Reject a position if another queen exists in the same column or diagonal.

Symmetry elimination: First-row choices are restricted to half the board; reflected solutions need not be generated separately.

Q92. N-Queens Problem with Column Restrictions

Idea: Some columns cannot contain queens. During backtracking, simply skip restricted columns.

Pseudocode

N_QUEENS_RESTRICTED(N, restricted)

Input:

N → board size restricted → set of columns where
queens are not allowed

Output:

valid queen arrangement or FAILURE

1. board[1..N] ← EMPTY
2. IF BACKTRACK(1) = TRUE THEN
3. RETURN board
4. ELSE
5. RETURN FAILURE
6. END IF

BACKTRACK(row)

1. IF row > N THEN

2. RETURN TRUE
3. END IF

4. FOR col $\leftarrow$ 1 TO N DO

5. IF col $\in$ restricted THEN
6. CONTINUE
7. END IF

8. IF SAFE(row, col) THEN
9. board[row] $\leftarrow$ col

10. IF BACKTRACK(row + 1) = TRUE THEN
11. RETURN TRUE
12. END IF

13. board[row] $\leftarrow$ EMPTY
14. END IF

15. END FOR

16. RETURN FALSE

SAFE(row, col)

1. FOR i $\leftarrow$ 1 TO row - 1 DO
2. IF board[i] = col THEN
3. RETURN FALSE
4. END IF

5. IF |board[i] - col| = |i - row| THEN
6. RETURN FALSE

7. END IF
8. END FOR

9. RETURN TRUE

Pruning:

- Skip restricted columns.
- Reject same-column positions.
- Reject diagonal conflicts.

If no branch succeeds, return **FAILURE**.

Q93. Sudoku Solver with Maximum Number Constraint per Row

Idea: Fill empty cells using backtracking. A number is accepted only if it satisfies **Sudoku rules** and does not make the row sum exceed its maximum limit.

Pseudocode

SUDOKU_WITH_ROW_LIMIT(grid, limit)

Input: grid[9][9] → partially filled Sudoku

limit[1..9] → maximum allowed sum for each row

Output:

completed Sudoku grid or FAILURE

1. IF SOLVE(0) = TRUE THEN
2. RETURN grid
3. ELSE
4. RETURN FAILURE
5. END IF

SOLVE(pos)

1. IF pos = 81 THEN
2. RETURN TRUE

```

3.      END IF

4.      row  $\leftarrow$  pos / 9
5.      col  $\leftarrow$  pos MOD 9

6.      IF grid[row][col]  $\neq$  0 THEN
7.      RETURN SOLVE(pos + 1)
8.      END IF

9.      FOR num  $\leftarrow$  1 TO 9 DO
10.     IF VALID(row, col, num) THEN

11.         newSum  $\leftarrow$  current row sum + num

12.         IF newSum  $\leq$  limit[row] THEN
13.         grid[row][col]  $\leftarrow$  num

14.         IF SOLVE(pos + 1) = TRUE THEN
15.         RETURN TRUE
16.         END IF

17.         grid[row][col]  $\leftarrow$  0
18.         END IF
19.     END IF
20. END FOR

21.     RETURN FALSE

```

VALID(row, col, num)

```

1.  IF num exists in row THEN

```

2. RETURN FALSE
3. IF num exists in column THEN
4. RETURN FALSE
5. IF num exists in 3×3 subgrid THEN
6. RETURN FALSE
7. RETURN TRUE

Pruning: Reject a number immediately when:

- it violates row uniqueness,
- it violates column uniqueness,
- it violates 3×3 subgrid uniqueness,
- row sum exceeds its maximum.

Q94. Sudoku Solver with Fixed Order of Empty Cells

Idea: The empty cells are already given in a specific order. The algorithm must fill them **in exactly that order**.

Pseudocode

SUDOKU_FIXED_ORDER(grid, emptyCells)

Input: grid[9][9] → partially filled Sudoku

emptyCells[] → predefined ordered list of empty cells

Output:

completed Sudoku grid or FAILURE

1. IF SOLVE(0) = TRUE THEN
2. RETURN grid
3. ELSE
4. RETURN FAILURE
5. END IF

SOLVE(index)

1. IF index = length(emptyCells) THEN
2. RETURN TRUE
3. END IF

4. (row, col) $\leftarrow$ emptyCells[index]

5. FOR num $\leftarrow$ 1 TO 9 DO

6. IF VALID(row, col, num) THEN
7. grid[row][col] $\leftarrow$ num

8. IF SOLVE(index + 1) = TRUE THEN
9. RETURN TRUE
10. END IF

11. grid[row][col] $\leftarrow$ 0
12. END IF

13. END FOR

14. RETURN FALSE

VALID(row, col, num)

1. IF num exists in row THEN
2. RETURN FALSE

3. IF num exists in column THEN
4. RETURN FALSE

5. IF num exists in 3 $\times$ 3 subgrid THEN
6. RETURN FALSE

7. RETURN TRUE

Pruning: Any candidate violating row, column, or subgrid rules is rejected immediately.

Important: Unlike normal Sudoku backtracking, the algorithm **does not choose the next empty cell**; it follows the given emptyCells order.

Q95. Subset Sum with Distinct Element Usage

Idea: Each occurrence of an element can be used **at most once**. When duplicate values exist, skip duplicates at the same recursion level to avoid generating the same subset repeatedly.

Pseudocode

SUBSET_SUM(A, target)

Input:

A[] → multiset of integers

target → required sum

Output:

TRUE and valid subset, or FALSE

1. Sort A
2. subset ← empty list

3. IF BACKTRACK(0, 0) = TRUE THEN
4. RETURN TRUE, subset
5. ELSE
6. RETURN FALSE
7. END IF

BACKTRACK(start, currentSum)

1. IF currentSum = target THEN
2. RETURN TRUE

```
3.     END IF

4.     IF currentSum > target THEN
5.         RETURN FALSE
6.     END IF

7.     FOR i ← start TO length(A) - 1 DO

8.         IF i > start AND A[i] = A[i-1] THEN
9.             CONTINUE
10.        END IF

11.        subset.add(A[i])

12.        IF BACKTRACK(i + 1, currentSum + A[i]) = TRUE THEN
13.            RETURN TRUE
14.        END IF

15.        subset.removeLast()

16.    END FOR

17.    RETURN FALSE Example
```

$A = \{2, 3, 3, 5\}$

Target = 8

Possible subset = $\{3, 5\}$